

Object-Oriented Programming(OOP)

1. What are object-oriented concepts? What is the difference between object-based, object-oriented, and fully object-oriented languages?

Object-Oriented Programming (OOP) is a way of organizing code using **objects** and **classes**. The four main principles of OOP are:

1. Encapsulation

Definition: Encapsulation is the process of bundling data (variables) and methods (functions) that operate on the data into a single unit (class). It restricts direct access to some details to ensure data security.

Real-Life Example:

Think of a **capsule medicine**. It contains multiple components (medicines), but you can't see or access them directly; you only take the capsule. Similarly, in programming, variables are hidden inside a class and accessed only through methods.

2. Abstraction

Definition: Abstraction means **hiding** the complex details and only showing the necessary functionality to the user.

Real-Life Example:

When you **drive a car**, you use the steering wheel, accelerator, and brakes. You **don't** need to understand the internal mechanics (engine, fuel combustion, etc.). The complex details are hidden, and only essential operations are exposed.

3. Inheritance

Definition: Inheritance allows one class to **acquire** the properties and methods of another class. It promotes code reuse.

Real-Life Example:

A **child inherits** traits from parents (like eye color, height, etc.), but they can also have their unique features.

4. Polymorphism

Definition: Polymorphism means **one name, multiple forms**—a function or method behaves differently based on the object calling it.

Real-Life Example:

A **person** can be a **student at school**, a **customer at a shop**, and a **passenger in a bus**—same person, different roles.

Types of Polymorphism:

1. **Method Overloading (Compile-time polymorphism):** Same method name, different parameters.

2. **Method Overriding (Runtime polymorphism):** Same method name in parent and child class, but different behavior.

Difference Between Object-Based, Object-Oriented, and Fully Object-Oriented Languages

Type	Description	Real-Life Example	Programming Languages
Object-Based Language	Uses objects but does not support inheritance and polymorphism.	A blueprint-only home design (cannot inherit features).	JavaScript (before ES6), VBScript
Object-Oriented Language	Supports all OOP features but also allows procedural programming.	A hybrid car (uses both fuel & electricity).	C++, Java
Fully Object-Oriented Language	Everything is treated as an object , strictly following OOP principles.	A pure electric car (runs only on electricity, no fuel).	Smalltalk, Ruby, Python

Key Differences:

- **Object-Based languages** allow objects but lack **inheritance and polymorphism**.
- **Object-Oriented languages** support OOP but allow procedural programming (like `main()` function in C++).
- **Fully Object-Oriented languages** strictly treat **everything** as an object (even primitive types like `int` are objects in Python).

2. What are the advantages of object-oriented programming?

1. Reusability (Code Reusability)

- OOP allows code reuse through inheritance, reducing duplication and improving maintainability.
- Example: A vehicle class can define common attributes (wheels, engine, speed), and car, bike, and truck classes can inherit from it without rewriting the same code.

2. Maintainability & Scalability

- Since OOP follows the modular approach (dividing code into classes and objects), it's easier to update and scale applications.
- Example: In an E-commerce application, if we need to add a new payment method, we can simply extend the Payment class without modifying the entire system.

3. Security (Data Hiding & Encapsulation)

- OOP ensures data security by making attributes private and allowing access only through getter & setter methods.
- Example: A bank account class hides the balance and allows transactions only through authorized methods.

4. Abstraction (Reducing Complexity)

- OOP hides complex implementation details and exposes only the **necessary** parts.
- **Example:** When using an **ATM machine**, users only see options like "Withdraw" and "Check Balance"; the internal workings remain hidden.

5. Polymorphism (Flexibility & Extensibility)

- One function can have **multiple behaviors**, allowing for more **flexible** and **scalable** applications.
- **Example:** A **printer** can print **documents, PDFs, images**, but the `print()` function behaves differently based on the input type.

6. Improved Productivity & Readability

- OOP follows a **structured approach**, making code easier to read, understand, and collaborate on.
- **Example:** Large-scale projects like **social media platforms (Facebook, LinkedIn, Instagram)** use OOP for better management and collaboration among developers.

3. What are classes and objects? Provide a real-life example.

A class is a template that defines the structure and behavior of objects. It contains data members (attributes) and methods (functions) that define an object's behavior.

Example: Car Class

A Car has properties like brand, color, speed and behaviors like `start()` and `stop()`.

An object is an instance of a class, representing a real-world entity with its own values for the defined attributes.

Here, `myCar` is an object of the Car class with specific values.

4. What are the characteristics of an object? Explain them.

1. Identity

- Every object has a unique identity, distinguishing it from other objects, even if they have the same properties.
- In Java, object identity is defined by its memory address (reference).

✅ Real-Life Example:

Even if two cars of the same brand and model exist, each has a unique registration number (identity).

2. State (Attributes/Properties)

- The state of an object is defined by its attributes (variables).
- These attributes store data related to the object, which may change over time.

✅ Real-Life Example:

A Mobile Phone has attributes like brand, model, battery percentage, storage capacity, etc.

3. Behavior (Methods/Functions)

- Behavior defines what an object can do (its functions or actions).
- It is implemented using methods in a class.

✅ Real-Life Example:

A Fan has behaviors like turn on, turn off, change speed

5. Why are getter and setter functions needed in a class?

Getter and setter methods are essential for controlling access to the private fields of a class. They provide a controlled interface to access and modify the values of private attributes while adhering to the principles of encapsulation and data validation in Object-Oriented Programming (OOP). Here's why they are necessary:

1. Encapsulation (Data Hiding)

- In OOP, encapsulation refers to the practice of restricting direct access to the internal state of an object. Private fields are used to keep data secure from external modification.
- Getter and setter methods allow controlled access to these private fields while keeping the details of how the data is stored hidden.

✅ Real-Life Example:

Imagine a bank account system where the balance should not be directly accessed or changed by anyone. Instead, users interact with the balance through controlled methods (getters and setters) to ensure safety.

2. Data Validation

- Setters allow you to include validation logic to check that the data being set meets certain conditions before updating the field.

- This ensures that the object's state remains consistent and valid throughout its lifecycle.
- Reason Explanation

Encapsulation :Protects private data from direct access.

Data Validation: Ensures only valid data is set.

Flexibility :Allows you to change internal logic without affecting the external interface.

Maintainability :Easier to update and maintain the code.

6. What are abstraction and encapsulation? Provide real-life examples.

Abstraction

Abstraction is the concept of hiding the complex implementation details of a system and exposing only the necessary features or functionality to the user. It allows users to interact with an object at a higher level without needing to understand its internal workings.

Purpose:

- It helps in simplifying the interaction with objects by providing a simple interface.
- It focuses on what the object does, rather than how it does it.

Real-Life Example of Abstraction:

Think of a car. When you drive, you interact with the car's steering wheel, accelerator, and brakes. These are the abstracted features exposed to you. However, you don't need to know the complex engine mechanics, fuel systems, or electronics that make the car work. The car's internal details are abstracted away, allowing you to focus only on the basic functions necessary for driving.

In programming, abstraction can be achieved using abstract classes or interfaces, where you define methods but don't necessarily provide the implementation in the base class.

Encapsulation

Encapsulation is the concept of bundling the data (variables) and the methods (functions) that operate on the data into a single unit, typically a class. It also involves restricting access to certain components (variables) by making them private and exposing them through public methods (getters and setters).

Purpose:

- It helps in data security and integrity.
- It provides a controlled way to access or modify the internal state of an object.

Real-Life Example of Encapsulation:

Think of a television remote control. You interact with the remote using buttons like Power, Volume Up, Channel Change, etc. These buttons represent the methods that control the remote's functionality. However, the internal circuits and mechanisms that process your button presses are hidden (encapsulated) from you.

In programming, encapsulation ensures that you can hide the internal details of an object and only expose the necessary functions to the user.

7. What is polymorphism? What are its types? Explain them with examples.

Polymorphism is one of the core concepts of Object-Oriented Programming (OOP). The term "polymorphism" comes from Greek, meaning "many shapes" or "many forms." It allows objects of different classes to be treated as objects of a common superclass, with the ability to take on many forms (or behaviors). Polymorphism enables a single interface to be used for different data types, which helps in reducing complexity and increasing flexibility.

In simple terms, polymorphism allows you to perform the same action in different ways depending on the context. It is achieved through method overriding (runtime polymorphism) or method overloading (compile-time polymorphism).

Types of Polymorphism

Polymorphism is generally classified into two main types:

1. Compile-time Polymorphism (Static Polymorphism)
2. Runtime Polymorphism (Dynamic Polymorphism)

1. Compile-time Polymorphism (Static Polymorphism)

Compile-time polymorphism occurs when the method to be invoked is determined at the time of compilation. This type of polymorphism is achieved through method overloading and operator overloading (though operator overloading is not allowed in some languages like Java).

Method Overloading:

In method overloading, multiple methods have the same name but differ in the number or types of parameters. The correct method is determined by the compiler based on the number and types of arguments passed.

Real-life Example of Method Overloading: Imagine a calculator that can perform the same operation (e.g., addition) on different data types — integers, floats, etc.

2.Runtime polymorphism occurs when the method to be invoked is determined at **runtime** based on the object type, rather than the reference type. This is achieved through **method overriding**, where a subclass provides its own implementation of a method that is already defined in the superclass.

Method Overriding:

In method overriding, a subclass provides a specific implementation of a method that is already defined in its superclass. The method in the subclass has the same signature (name, return type, and parameters) as the method in the superclass. When a method is called on a superclass reference that points to a subclass object, the subclass's version of the method is invoked.

Real-life Example of Method Overriding:

Consider a **vehicle** that can start, but different types of vehicles (car, bike, etc.) have different ways to start.

8. What is function overloading? What are the rules of function overloading? Why is the return type not considered in function overloading?

Function Overloading is a feature in Object-Oriented Programming (OOP) that allows multiple functions (methods) to have the same name but differ in the **number** or **type** of parameters. The compiler differentiates between these functions based on their method signature (method name and parameter list).

Function overloading is commonly used to perform similar operations on different types or numbers of inputs without changing the method name.

Real-life Example:

Think of a **printer** that can print documents. The printer can print a **single page** or multiple **pages**, and the method name is the same, but the number of pages (parameter) varies.

Rules of Function Overloading:

Method Name Must Be Same: The method name must be identical in all overloaded methods.

Different Parameter List: The method signature must differ in the number or types of parameters. The **return type does not count** for overloading purposes.

Parameter Type or Order Can Differ: You can overload methods based on different **parameter types** or **their order**.

Return Type Does Not Matter: The return type of the methods cannot be used to distinguish overloaded methods. In other words, if two methods differ only by their return type, it is **not considered overloading**.

The **return type** is **not considered** when overloading methods because the method signature, which is used to differentiate overloaded methods, includes only the **method name** and **parameter list**. The return type is not part of the method signature. If the return type were considered, it could create ambiguity when the method is called, making it difficult to determine which overloaded method to invoke.

Example of Ambiguity:

Imagine a scenario where we have two methods with the same parameters but different return types. The compiler would have no way of knowing which method to invoke based only on the return type, leading to confusion.

9. What are the different types of hierarchy? When should each be used?

In Object-Oriented Programming (OOP), **hierarchy** refers to the relationship between classes and their objects. Hierarchies define how different classes relate to each other, either as more specific (subclasses) or more general (superclasses). There are different types of hierarchies that are commonly used to design systems and model real-world relationships:

1. Inheritance Hierarchy:

- **Description:** In an inheritance hierarchy, classes are arranged based on **parent-child** relationships. A class inherits from a superclass (parent), which means it can access the properties and methods of the superclass. The subclass (child) can also add new properties or override methods from the superclass.
- **When to Use:**
 - **Is-a Relationship:** Use inheritance when a subclass "is a" type of its superclass. For example, a Dog class **is a** type of Animal.
 - **Code Reusability:** When you want to reuse code from a parent class in multiple child classes.
 - **Specialization:** When the child class specializes or extends the functionality of the parent class.

2. Composition Hierarchy (Has-a Relationship):

- **Description:** In composition, one class contains objects of other classes as its members. This represents a **has-a** relationship, where the contained class is a part of

the parent class, and the lifecycle of the contained object is tied to the lifecycle of the parent class.

- **When to Use:**
 - **Has-a Relationship:** Use composition when one class needs to contain another class as part of its state. For example, a `Car` **has-a** `Engine`.
 - **Strong Encapsulation:** When you want to encapsulate the implementation of a class and provide more flexibility (i.e., composition allows changing or swapping components more easily than inheritance).
 - **Lifetime Dependency:** When the contained object is strongly bound to the lifecycle of the parent object (the child object cannot exist without the parent).

3. Aggregation Hierarchy:

- **Description:** Aggregation is a specialized form of association where one class **contains** or **owns** objects of other classes, but the contained objects can exist independently of the parent object. In aggregation, the lifecycle of the contained object is **not** managed by the parent class.
- **When to Use:**
 - **Has-a Relationship:** Similar to composition, aggregation also models the **has-a** relationship but with a weaker connection.
 - **Independence:** Use aggregation when the contained objects can exist independently of the parent object. For example, a `Team` **has** `Players`, but the players can exist independently of the team.

4. Interface Hierarchy:

- **Description:** In an interface hierarchy, interfaces are organized based on the idea of defining multiple behaviors that can be implemented by various classes. An interface can extend other interfaces, creating a multi-level interface hierarchy.
- **When to Use:**
 - **Multiple Behavior Inheritance:** Use interface hierarchy when you need to define multiple behaviors for different classes that may not share a common parent class.
 - **Decoupling:** When you want to decouple the implementation from the interface to allow flexibility and adherence to SOLID principles (especially Interface Segregation Principle).

5. Multiple Inheritance Hierarchy (via Interfaces in Java):

- **Description:** Multiple inheritance occurs when a class can inherit from more than one class. While most OOP languages (including Java) do not allow multiple inheritance with classes, interfaces provide a way to achieve this.
- **When to Use:**
 - **Behavioral Inheritance:** Use multiple inheritance when a class needs to inherit behavior from multiple sources. In Java, this is achieved through interfaces (since Java does not allow multiple class inheritance).
 -

10. What is the difference between function overloading and function overriding?

1. Function Overloading

Definition:

Function overloading occurs when two or more functions in the same class have the same name but differ in the number or type of their parameters.

- **Purpose:** To allow the creation of multiple functions with the same name but different parameters, making the code easier to understand and use.
- **Compile-time Polymorphism:** Overloading is resolved at compile-time, meaning the decision about which method to call is made by the compiler based on the method signature.

2. Function Overriding

Definition:

Function overriding occurs when a subclass provides its own specific implementation of a method that is already defined in its superclass. The method in the subclass must have the same name, return type, and parameters as the method in the superclass.

- **Purpose:** To modify or extend the functionality of a method inherited from a parent class in a subclass.
- **Run-time Polymorphism:** Overriding is resolved at runtime, meaning the decision about which method to call is made based on the object type (i.e., the instance of the class being used).

11. Why is the constructor considered a special member function of the class?

A **constructor** is a **special member function** of a class because:

Automatic Invocation – It is called automatically when an object is created.

Same Name as Class – The constructor name must match the class name.

No Return Type – Unlike regular functions, constructors do not have a return type.

Initializes Objects – It sets up initial values for an object's attributes.

Can Be Overloaded – Multiple constructors with different parameters can exist in a class.

Not Inherited – Constructors are not inherited but can be accessed using `super()`.

Ensures Object Integrity – Prevents uninitialized or invalid object states.

Supports Default & Parameterized Forms – If no constructor is defined, the compiler provides a default one.

A constructor is considered a special member function because it plays a vital role in object creation, has a unique syntax (same name as the class, no return type), is automatically invoked during object instantiation, and ensures proper initialization of objects. These characteristics distinguish constructors from other regular methods in a class

12. What is object slicing? Explain object slicing in the context of upcasting.

Object slicing occurs when an object of a derived class is assigned to an object of its base class, causing the derived class's specific attributes (extra members) to be lost. This happens because the base class object can only hold base class members, not the additional members introduced in the derived class.

Explanation in the Context of Upcasting

In upcasting, a base class pointer or reference is used to refer to a derived class object. Unlike object slicing, this preserves all data because the actual object remains of the derived type

13. What is downcasting, and when is it required?

Downcasting is the process of converting a parent class reference into a child class reference. It is required when you want to access child-specific methods or properties that are not present in the parent class.

When is Downcasting Required?

- When you need to access child class-specific methods that are not in the parent class.
- When working with polymorphism, where a method returns a parent type but contains a child object.

14. What are association, composition, and aggregation? Explain with examples.

Association is a relationship between two classes where one class is connected to another. It is a broad term that includes Aggregation and Composition, which define different levels of dependency between objects.

1. Association

- It represents a relationship between two independent classes where they communicate with each other.
- It can be one-to-one, one-to-many, many-to-one, or many-to-many.

Example: A Student and a Teacher

A student and a teacher are related, but they can exist independently.

2. Aggregation (Weak Association)

- A "Has-A" relationship where the child object can exist independently of the parent.
- The parent contains the child but does not own it.
- Example: Department and Faculty
- A Department has multiple Faculties, but if the department is removed, faculties still exist.

3. Composition (Strong Association)

- A stronger form of Aggregation where the child object cannot exist without the parent.
- If the parent is destroyed, the child also gets destroyed.

Example: Car and Engine

A Car has an Engine, but an Engine alone is useless without a Car.

15. What are the different types of inheritance? Explain with examples. What are the problems with multiple inheritance?

Single Inheritance

- In single inheritance, a subclass derives from a single superclass.
- The child class inherits all non-private members of the parent class.

♦ Real-life Example:

Consider a parent-child relationship where a child inherits certain traits from the parent, such as eye color or height.

Multilevel Inheritance

- A derived class (child) inherits from another derived class (parent).
- This forms a chain of inheritance.

♦ Real-life Example:

A grandfather → father → child relationship. A child inherits traits from both the father and grandfather.

Hierarchical Inheritance

- Multiple child classes inherit from a single parent class.

♦ Real-life Example:

A vehicle can have different types like car and bike, both having common properties but behaving differently.

Multiple Inheritance (Not Supported in Java with Classes)

Java does not support multiple inheritance with classes due to the Diamond Problem. However, it is possible using interfaces.

◆ Real-life Example:

A student can be both an athlete and a musician. But if both trainers (music and sports) give different instructions, there is confusion.

The Diamond Problem occurs when a class inherits from two classes that have the same method, leading to ambiguity.

◆ Example:

If Class B and Class C both inherit from Class A, and Class D inherits from both B and C, then Java wouldn't know which method to execute.

16. What are the different types of design patterns? Explain the singleton design pattern.

Design patterns are proven solutions to common software design problems. They help improve code reusability, maintainability, and scalability.

Categories of Design Patterns

Design patterns are classified into three main types:

Type	Description	Examples
Creational Patterns	Focus on object creation mechanisms to increase flexibility and reuse	Singleton, Factory, Abstract Factory, Builder, Prototype
Structural Patterns	Focus on composing objects and classes to form larger structures	Adapter, Bridge, Composite, Decorator, Facade, Proxy, Flyweight
Behavioral Patterns	Focus on communication between objects and class interactions	Observer, Strategy, Command, Chain of Responsibility, Template Method, Mediator

Core Java

1. What is a wrapper class? What is its use?

Answer:

A wrapper class is a class in Java that wraps primitive data types into objects. Each primitive type has a corresponding wrapper class, such as Integer for int, Double for double, etc.

Uses:

- Provides utility methods for conversions.
 - Allows primitives to be used in collections (as collections work with objects).
 - Supports autoboxing and unboxing.
-

2. What are the methods of the java.lang.Object class? Which are the native methods?

Answer:

The Object class has the following methods:

- equals(Object obj)
- hashCode()
- toString()
- clone()
- finalize()
- getClass()
- notify(), notifyAll(), and wait() (three overloaded versions)

Native methods:

- hashCode()
 - clone()
 - wait()
 - notify()
 - notifyAll()
-

3. What is the need for packages? What types are allowed to be declared in a package?

Answer:

Need for packages:

- **Structured Code:** Packages help in organizing classes and interfaces into a manageable structure. This modularity makes code easier to navigate and maintain, which is crucial in large-scale applications.
- **Avoiding Naming Conflicts:**

Unique Namespace: Packages provide a unique namespace for each set of classes, preventing naming conflicts. This is particularly important in large projects with multiple teams working on different modules that might use similar class names.

- **Access Control and Encapsulation:**

Controlled Access: Using access modifiers like public, protected, and private in conjunction with packages, developers can control the visibility of classes and their members. This helps in enforcing encapsulation and protecting the internal implementation of classes.

Package-Private Access: The default access level (no modifier) allows classes and members to be accessed only within the same package, offering an additional layer of control.

Types allowed in a package:

- Classes
 - Interfaces
 - Enumerations
 - Annotations
-

4. Why can we not declare multiple public classes in a single .java file?

Answer:

Java enforces that each .java file can have only one public class because:

- The file name must match the public class name.
 - Having multiple public classes would create ambiguity in compilation.
-

5. What is the difference between import and static import?

Answer:

- import: Used to import a class or package. Example: `import java.util.ArrayList;`
- static import: Used to import static members of a class, allowing them to be used without prefixing the class name. Example:

```
import static java.lang.Math.PI;
```

```
System.out.println(PI); // No need to write Math.PI
```

6. How can we pass an argument to a method by reference? Explain with an example.

Answer:

Java does not support passing by reference, but objects are passed by reference internally. Example:

```
class Test {  
    int num;  
}  
  
class Demo {  
    static void modify(Test obj) {  
        obj.num = 10;  
    }  
  
    public static void main(String[] args) {  
        Test t = new Test();  
        t.num = 5;  
        modify(t);  
        System.out.println(t.num); // Output: 10  
    }  
}
```

7. What is the difference between checked and unchecked exceptions?**Answer:**

- **Checked Exceptions:** Must be handled using try-catch or declared using throws. Example: IOException, SQLException.
 - **Unchecked Exceptions:** Occur at runtime and do not require explicit handling. Example: NullPointerException, ArithmeticException.
-

8. What are the advantages and disadvantages of generics?**Answer:****Advantages:****1. Type Safety:**

- **Compile-Time Checking:** Generics allow for type checking at compile-time, reducing the risk of runtime errors. For example, if you try to add an object of the wrong type to a generic collection, the compiler will catch the error.

2. Code Reusability:

- **Reusable Code:** Generics enable you to write more reusable and general code. You can create classes, methods, and interfaces that work with any type, rather than being limited to specific types.

3. Elimination of Type Casting:

- **Less Casting:** When you retrieve elements from a generic collection, you don't need to perform explicit type casting. This makes the code cleaner and more readable.

Disadvantages:

1. Complexity:

- **Learning Curve:** Generics can be complex and may introduce a steep learning curve for beginners. Understanding bounded types, wildcards, and type erasure can be challenging.

2. Type Erasure:

- **Limited Access to Type Information:** Due to type erasure, the generic type information is removed at runtime. This means you can't perform certain operations that rely on runtime type information.

3. Cannot Use Primitives:

- **Object Requirement:** Generics work only with objects, not primitive types. This means you have to use wrapper classes (e.g., `Integer` instead of `int`), which can lead to performance overhead due to boxing and unboxing.

4. Limited Support for Generics in Arrays:

- **Array Restrictions:** You cannot create arrays of generic types directly. This can be restrictive and requires workarounds, such as using `List` instead of arrays.

9. What is the difference between Comparable and Comparator?

Answer:

- **Comparable:** Implemented in the class itself using `compareTo()`. Used for natural ordering.
- **Comparator:** Defined externally to compare objects in different ways using `compare()`.

Example:

```
class Student implements Comparable<Student> {  
    int age;  
    public int compareTo(Student s) {  
        return this.age - s.age;  
    }  
}
```

10. What is the difference between ArrayList and Vector? How do we use ListIterator?

Answer:

1. Synchronization:

- **ArrayList:** It is not synchronized, meaning it is not thread-safe. If multiple threads access an ArrayList concurrently, and at least one thread modifies it, there is no guarantee of consistency without external synchronization.
- **Vector:** It is synchronized, meaning it is thread-safe. All methods in Vector are synchronized, which ensures consistency when accessed by multiple threads simultaneously. However, this synchronization comes with a performance cost.

2. Performance:

- **ArrayList:** Due to the lack of synchronization, ArrayList generally performs faster than Vector in single-threaded environments.
- **Vector:** The synchronization overhead makes Vector slower compared to ArrayList for single-threaded operations.

3. Growth:

- **ArrayList:** Increases its size by 50% of the current array size when it runs out of space.
- **Vector:** Doubles its size when it runs out of space, meaning it grows by 100%.

4. Legacy:

- **ArrayList:** Part of the java.util package introduced in Java 1.2 and is widely used in modern Java applications.
- **Vector:** Part of the original Java 1.0 and is considered a legacy class. While still supported, it is generally discouraged in favor of ArrayList.

Using ListIterator:

```
List<String> list = new ArrayList<>();
```

```
ListIterator<String> itr = list.listIterator();  
while (itr.hasNext()) {  
    System.out.println(itr.next());  
}
```

11. What is a fail-fast and fail-safe iterator?

Answer:

- **Fail-Fast Iterator:** Throws `ConcurrentModificationException` if the collection is modified while iterating. Example: `ArrayList`.
 - **Fail-Safe Iterator:** Does not throw an exception as it operates on a cloned collection. Example: `CopyOnWriteArrayList`.
-

12. What are serialization and deserialization? What is the significance of `SerialVersionUID`?

Answer:

- **Serialization:** Converting an object into a byte stream.
- **Deserialization:** Converting a byte stream back into an object.
- **SerialVersionUID** ensures version compatibility of serialized objects across different versions of the class.

Example:

```
class Employee implements Serializable {  
    private static final long serialVersionUID = 1L;  
}
```

13. What is the relationship between the Thread's `start()` and `run()` methods?

Answer:

1. `start()` Method:

- **Purpose:** The `start()` method is used to initiate a new thread of execution. When you call `start()`, the Java Virtual Machine (JVM) creates a new thread and schedules it to run. This method ensures that the `run()` method is executed in a new thread, separate from the thread that called `start()`.

- **Behavior:** Calling start() does not immediately run the run() method. Instead, it puts the thread in a "ready-to-run" state, and the JVM decides when the thread gets CPU time.

2. run() Method:

- **Purpose:** The run() method contains the code that defines the thread's task. This is where you place the logic that you want the thread to execute.
- **Behavior:** The run() method is called internally by the start() method. You should not call run() directly if you want the code to run in a separate thread.

Example:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // Starts a new thread
        t.run(); // Runs in the main thread, not as a separate thread
    }
}
```

14. When should we create a thread by implementing Runnable and by extending the Thread class?

Implementing Runnable (Preferred Approach):

- When you need to extend another class (since Java doesn't support multiple inheritance).
- When you want better separation of task (Runnable) and execution mechanism (Thread).
- When sharing the same Runnable instance across multiple threads is needed.

15. What is a functional interface? Which functional interfaces are predefined, and where are they used?

A functional interface is an interface that has exactly one abstract method (but can have multiple default or static methods). It enables Lambda Expressions and Functional Programming in Java.

Predefined Functional Interfaces (in `java.util.function` package):

- **Predicate<T>** – Takes T, returns boolean. Used for filtering.

```
Predicate<Integer> isEven = num -> num % 2 == 0;
```

```
System.out.println(isEven.test(4)); // true
```

- **Function<T, R>** – Takes T, returns R. Used for transformations.

```
Function<String, Integer> length = str -> str.length();
```

```
System.out.println(length.apply("Java")); // 4
```

- **Consumer<T>** – Takes T, returns nothing. Used for processing elements.

```
Consumer<String> print = System.out::println;
```

```
print.accept("Hello");
```

- **Supplier<T>** – Takes nothing, returns T. Used for fetching values.

```
Supplier<Double> random = Math::random;
```

```
System.out.println(random.get());
```

- **BiFunction<T, U, R>** – Takes two arguments, returns a value.

```
BiFunction<Integer, Integer, Integer> sum = (a, b) -> a + b;
```

```
System.out.println(sum.apply(3, 5)); // 8
```

Where Used?

- Stream API (filter, map, etc.)
- Lambda expressions
- Event handling in GUI frameworks

16. What is the significance of filter(), map(), flatMap(), and reduce() operations? In which scenarios are they used?

filter() – Used for filtering elements based on a condition.

- Takes a Predicate<T> and returns a stream with matching elements.

map() – Transforms each element using a function.

- Takes a Function<T, R> and applies it to each element.

flatMap() – Flattens multiple nested collections into a single stream.

- Used when each element itself is a collection.

reduce() – Performs aggregation operations like sum, min, max, etc.

- Takes an identity value and a BinaryOperator function.
-

Java EE

1. Explain the Servlet and JSP lifecycle.

Servlet Lifecycle:

- 1) Loading & Instantiation: The servlet class is loaded into memory and instantiated by the servlet container.
- 2) Initialization (init() method): Called once to initialize the servlet.
- 3) Request Handling (service() method): Invoked for each client request. Calls doGet(), doPost(), etc. based on HTTP request type.
- 4) Destruction (destroy() method): Called once before the servlet is removed from memory.

JSP Lifecycle:

- 1) Translation: JSP is converted into a Servlet.
- 2) Compilation: The generated servlet is compiled into a .class file.
- 3) Initialization (jspInit() method): Called once when the JSP is initialized.
- 4) Request Handling (_jspService() method): Handles client requests.
- 5) Destruction (jspDestroy() method): Called before the JSP is removed.

2. What is the difference between a Redirect and a RequestDispatcher scenario?

Feature	Redirect (sendRedirect())	RequestDispatcher (forward())
Server/Client	Client-side	Server-side
URL Change	Yes	No
Performance	Slower (New request)	Faster (Same request)
When to Use	Redirecting to an external URL	Forwarding within the same application

3. What is session tracking? How is it implemented in Java?

Session tracking is the process of maintaining a user's state across multiple HTTP requests.

Ways to implement session tracking:

1. **Cookies:** Small text files stored in the browser.
2. **URL Rewriting:** Session ID is appended to the URL.
3. **Hidden Form Fields:** Session data is stored in form fields.
4. **HTTP Session (HttpSession API):** Server-side session storage.

4. What is Hibernate? Explain the Hibernate architecture.

Hibernate is an ORM (Object-Relational Mapping) framework that simplifies database interactions by mapping Java objects to relational tables.

Hibernate Architecture Components:

1. **SessionFactory:** Manages sessions and provides database connections.
2. **Session:** Represents a unit of work with the database.
3. **Transaction:** Handles atomic operations.
4. **Query:** Used to execute HQL (Hibernate Query Language) or native SQL queries.
5. **Configuration:** Contains Hibernate settings (hibernate.cfg.xml).

6. Explain the Hibernate entity lifecycle.

In Hibernate, an entity goes through four lifecycle states:

1. **Transient:** Object is created but not associated with Hibernate.
 2. **Persistent:** Object is saved using session.save() and managed by Hibernate.
 3. **Detached:** Object is removed from Hibernate's session.
 4. **Removed:** Object is deleted using session.delete().
-

6. What is the difference between get() and load() in Hibernate?

Feature	get()	load()
Return Type	Actual object	Proxy object
When Used	Fetch immediately	Fetch lazily (Only when accessed)
If Not Found	Returns null	Throws ObjectNotFoundException

7. How can we call a stored procedure in Hibernate?

Using @NamedStoredProcedureQuery or Session.createStoredProcedureQuery().

Example:

```
StoredProcedureQuery query =  
session.createStoredProcedureQuery("getEmployeeById")  
        .setParameter("empld", 1);  
  
query.execute();
```

8. What is IoC and Dependency Injection?

Inversion of Control (IoC) is a design principle where object creation and dependency management are handled by the Spring container.

Dependency Injection (DI) is a technique where dependencies are injected rather than created inside the class.

9. What is autowiring? What are the types of autowiring?

Autowiring allows Spring to automatically inject dependencies.

Types of Autowiring:

1. **@Autowired on Field**
2. **@Autowired on Constructor**
3. **@Autowired on Setter**
4. **@Qualifier** – Used to resolve ambiguity.

10. Explain the Spring bean lifecycle.

1. **Instantiation** – Bean is created.
2. **Dependency Injection** – Dependencies are injected.
3. **@PostConstruct / init()** – Custom initialization.
4. **Ready for Use** – Bean is used by the application.
5. **@PreDestroy / destroy()** – Cleanup before shutdown.

11. What is the use of @Transactional? Why should it be used on the service layer?

@Transactional manages database transactions.

Why Service Layer?

- Ensures atomicity (all operations succeed or fail together).
- Prevents partial updates.

12. Explain the Spring MVC lifecycle.

1. Request received
2. DispatcherServlet identifies the controller
3. Controller processes the request
4. Service layer performs business logic
5. ViewResolver selects a view
6. Response is generated

13. What is the difference between SOAP and REST?

Feature	SOAP	REST
Protocol	XML-based	HTTP-based
Speed	Slower	Faster
Stateless	Stateful	Stateless

@RestController is used in REST APIs to indicate that the class returns JSON/XML instead of a view.

14. What is Spring Boot? What are opinionated defaults?

Spring Boot is a framework built on top of Spring that simplifies the development of standalone, production-ready Spring applications with minimal configuration. It eliminates boilerplate code and provides built-in features like:

- Auto-configuration (automatically configures beans based on dependencies)

- Embedded servers (Tomcat, Jetty, Undertow)
- Production-ready features (Actuator, logging, monitoring)

What are Opinionated Defaults?

Spring Boot follows a convention over configuration approach, meaning it provides predefined sensible defaults for common configurations to reduce developer effort.

For example:

- Embedded Tomcat as the default web server (instead of manually configuring one).
- H2 Database auto-configured if no database is specified.
- Spring MVC enabled automatically if spring-web is present.
- Logging (SLF4J + Logback) configured by default.

JDBC

1. What is JDBC, and why is it used?

JDBC (Java Database Connectivity) is an API that allows Java applications to interact with databases. It provides a standard interface for connecting to a relational database, executing queries, and retrieving results.

Why is it used?

- Platform-independent database connectivity.
- Allows executing SQL commands (CRUD operations).
- Supports transaction management.
- Works with various database drivers.

2. What are the different types of JDBC drivers?

JDBC drivers act as a bridge between Java applications and the database. There are four types:

1. Type-1: JDBC-ODBC Bridge Driver
 - Uses ODBC driver for database communication.
 - Slow and platform-dependent.
 - Deprecated in Java 8.
2. Type-2: Native-API Driver

- Uses vendor-specific client-side libraries.
 - Better performance than Type-1 but not fully portable.
 - 3. Type-3: Network Protocol Driver (Middleware Driver)
 - Uses a middleware server to communicate with the database.
 - More portable but adds network overhead.
 - 4. Type-4: Thin Driver (Pure Java Driver)
 - Communicates directly with the database using native protocol.
 - Most efficient and widely used in modern applications.
-

3. What are the core interfaces of JDBC?

JDBC consists of five key interfaces:

1. **Driver** – Handles communication with the database driver.
 2. **Connection** – Manages database connections.
 3. **Statement** – Executes SQL queries.
 4. **ResultSet** – Holds data retrieved from the database.
 5. **PreparedStatement** – Optimized SQL execution with parameterized queries.
-

4. How do you manage database connections in JDBC?

Connections are managed using the DriverManager or DataSource.

Using DriverManager:

```
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/dbname",  
"root", "password");
```

Using DataSource (Preferred in enterprise apps for connection pooling):

5. What is the difference between Statement, PreparedStatement, and CallableStatement?

Feature	Statement	PreparedStatement	CallableStatement
SQL Execution	Direct execution	Precompiled for efficiency	Used for stored procedures

Feature	Statement	PreparedStatement	CallableStatement
Performance	Low	Faster due to pre-compilation	Optimized for procedures
Security	Prone to SQL injection	Prevents SQL injection	Secure
Use Case	Simple queries	Queries with parameters	Stored procedures

6. How are transactions managed in JDBC?

Transactions in JDBC are controlled using **commit()** and **rollback()** on a **Connection** object.

Steps:

1. Disable auto-commit (setAutoCommit(false)).
2. Execute queries.
3. Commit changes (commit()) or roll back (rollback()) in case of an error.

7. What is the role of a ResultSet in JDBC?

ResultSet holds data retrieved from the database after executing a query.

Types of ResultSet:

1. **Forward-only (TYPE_FORWARD_ONLY)** – Moves forward only.
2. **Scrollable (TYPE_SCROLL_INSENSITIVE)** – Can move forward and backward.
3. **Updatable (CONCUR_UPDATABLE)** – Can update rows.

8. What are batch updates in JDBC?

4. Batch updates allow multiple SQL commands to be executed in a single request, improving performance.

9. What is the purpose of the DataSource interface in JDBC?

DataSource is used for connection pooling, providing better performance than **DriverManager**. It abstracts connection details and is used in enterprise applications.

10. How does JDBC handle SQL exceptions?

SQL exceptions are handled using **try-catch** blocks and **SQLException** methods like `getMessage()`, `getErrorCode()`, and `getSQLState()`.

11. What is connection pooling in JDBC, and how does it impact performance?

Connection pooling reuses database connections instead of creating new ones, significantly reducing overhead.

12. How do you handle SQL injection in JDBC-based applications?

SQL injection can be prevented using PreparedStatement, input validation, and escaping characters.

13. Explain the implementation of advanced retry logic for transient JDBC errors in large systems.

Retry logic ensures that transient failures (e.g., network timeouts, temporary DB unavailability) do not cause complete system failure.

MYSQL

1. What is MySQL? What are its key features?

MySQL is an open-source Relational Database Management System (RDBMS) based on SQL (Structured Query Language). It is widely used for web applications, data warehousing, and large-scale data processing.

Key Features:

- **Scalability:** Supports large databases with millions of rows.
- **High Performance:** Optimized storage engines like InnoDB.
- **Cross-Platform Support:** Works on Linux, Windows, and macOS.
- **ACID Compliance:** Ensures reliability with transactions.
- **Replication and Clustering:** Supports Master-Slave and Master-Master replication.
- **Stored Procedures & Triggers:** Enhances automation and efficiency.
- **Security:** User authentication and privilege management.

2. How do you create a database in MySQL?

```
CREATE DATABASE mydatabase;
```

```
USE mydatabase;
```

- The CREATE DATABASE statement creates a new database.
- The USE statement selects the database for operations.

3. What is the difference between CHAR and VARCHAR data types?

Feature	CHAR	VARCHAR
Storage	Fixed length	Variable length
Speed	Faster, as storage is fixed	Slightly slower, as it varies

Feature	CHAR	VARCHAR
Padding	Pads with spaces to maintain fixed length	No padding, saves space
Usage	When length is fixed (e.g., country codes)	When length varies (e.g., names)

Example:

```
CREATE TABLE example (
    col1 CHAR(10),
    col2 VARCHAR(10)
);
```

- CHAR(10) always stores 10 characters, adding spaces if necessary.
- VARCHAR(10) stores only the necessary characters.

4. How do you retrieve all rows from a table?

```
SELECT * FROM table_name;
```

This selects all columns and rows from a table.

5. What is the purpose of the WHERE clause in a SQL query?

The WHERE clause filters rows based on a condition.

```
SELECT * FROM employees WHERE salary > 50000;
```

Only retrieves employees with salaries greater than 50,000.

6. How do you add, delete, and modify a column in an existing table?

- **Add a column:**

```
ALTER TABLE employees ADD age INT;
```

- **Delete a column:**

```
ALTER TABLE employees DROP COLUMN age;
```

- **Modify a column:**

```
ALTER TABLE employees MODIFY COLUMN age VARCHAR(3);
```

6. What is the difference between PRIMARY KEY and UNIQUE constraints?

Primary Key:

- Ensures uniqueness and NOT NULL constraint (i.e., cannot have NULL values).
- A table can have only one primary key.
- Used to uniquely identify each record in a table.

- Creates an automatic index for faster searches.
- Unique Constraint:
 - Ensures uniqueness, but NULL values are allowed (except in some databases like MySQL with strict mode).
 - A table can have multiple UNIQUE constraints.
 - Prevents duplicate values in a column but does not enforce NOT NULL.

Example:

```
CREATE TABLE users (
    id INT PRIMARY KEY,
    email VARCHAR(100) UNIQUE
);
```

8. What are the various data types available in MySQL?

Numeric Data Types → Used for storing numbers.

- Integer Types (TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT) → Store whole numbers of different sizes.
 - Floating-Point Types (FLOAT, DOUBLE) → Used for decimal values but may have precision issues.
 - Fixed-Point Type (DECIMAL) → Ideal for financial calculations to maintain accuracy.
 - BIT Type (BIT(n)) → Stores binary values (e.g., BIT(8) for 8-bit storage).
 - ◆ *Example:* salary DECIMAL(10,2) stores numbers like 12345.67 accurately.
-

2.String (Character) Data Types → Used for storing text.

- Fixed Length (CHAR(n)) → Uses a fixed space, efficient for uniform-length data.
 - Variable Length (VARCHAR(n)) → Uses only the required space, great for names/descriptions.
 - Text Storage (TEXT, TINYTEXT, MEDIUMTEXT, LONGTEXT) → Stores large amounts of text.
 - Binary Storage (BLOB, TINYBLOB, MEDIUMBLOB, LONGBLOB) → Stores images/files in binary form.
 - Predefined Values (ENUM, SET) → Used for limited choices (e.g., ENUM('Male', 'Female')).
 - ◆ *Example:* name VARCHAR(50), gender ENUM('Male', 'Female').
-

3. Date & Time Data Types → Used for date and time storage.

- DATE → Stores date only (YYYY-MM-DD).
 - DATETIME → Stores date & time (YYYY-MM-DD HH:MM:SS).
 - TIMESTAMP → Auto-updates with current time.
 - TIME → Stores only time (HH:MM:SS).
 - YEAR → Stores only year (YYYY).
- ♦ Example: joining_date DATE, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP.

9. What is the purpose of the AUTO_INCREMENT attribute?

Automatically generates sequential values for a column.

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

The id column auto-increments with each new row.

10. How do you perform CRUD operations?

- **Create:** INSERT INTO table_name VALUES (...);
- **Read:** SELECT * FROM table_name;
- **Update:** UPDATE table_name SET column=value WHERE condition;
- **Delete:** DELETE FROM table_name WHERE condition;

11. What are JOINS? Explain types with examples.

- **Joins** are used to combine records from two or more tables based on a related column.
- **INNER JOIN:** Returns only the common records that exist in both tables.

ProductID	ProductName	CategoryID	Price
1	Chais	1	18
2	Chang	1	19
3	Aniseed Syrup	2	10

and a selection of the **Categories** table:

CategoryID	CategoryName	Description
1	Beverages	Soft drinks, coffees, teas, beers, and ales
2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
3	Confections	Desserts, candies, and sweet breads

- `SELECT ProductID, ProductName, CategoryName`
`FROM Products`
`INNER JOIN Categories ON Products.CategoryID = Categories.CategoryID;`
- **LEFT JOIN:** Returns all records from the left table and matching records from the right. If no match, NULL is returned.

`SELECT Products.ProductName, Categories.CategoryName`
`FROM Products`
`LEFT JOIN Categories ON`
`Products.CategoryID = Categories.CategoryID;`
- **RIGHT JOIN:** Returns all records from the right table and matching records from the left. If no match, NULL is returned.

`SELECT Products.ProductName, Categories.CategoryName`
`FROM Products RIGHT JOIN Categories`
`ON Products.CategoryID = Categories.CategoryID;`
- **FULL JOIN:** Returns all records from both tables, filling NULLs where there's no match.
(*Not supported in MySQL, but can be simulated using UNION.*) Example:

`SELECT Products.ProductName, Categories.CategoryName`
`FROM Products`
`LEFT JOIN Categories ON Products.CategoryID = Categories.CategoryID`
`UNION`
`SELECT Products.ProductName, Categories.CategoryName`
`FROM Products`
`RIGHT JOIN Categories ON Products.CategoryID = Categories.CategoryID;`

12. How does GROUP BY work?

The GROUP BY clause in SQL is used to group rows that have the same values in specified columns into summary rows, such as counting the number of rows in each group. It is often used with aggregate functions (like COUNT(), SUM(), AVG(), MAX(), MIN()) to perform calculations on each group of rows.

Syntax:

```
SELECT column1, aggregate_function(column2)
FROM table
```

GROUP BY column1;

Ex:

SELECT ProductName, SUM(Quantity) AS TotalQuantity

FROM Sales

GROUP BY ProductName;

13. What is a foreign key?

A **foreign key** is a column or a set of columns in one table that establishes a link between the data in two tables. It enforces referential integrity by ensuring that the value in the foreign key column must match a primary key value in another table.

CREATE TABLE orders (

id INT PRIMARY KEY,

customer_id INT,

FOREIGN KEY (customer_id) REFERENCES customers(id)

);

14. Difference between INNER, LEFT, RIGHT, and FULL OUTER JOIN?

- INNER JOIN: Only matching rows.
- LEFT JOIN: All from left + common.
- RIGHT JOIN: All from right + common.
- FULL JOIN: All rows from both.

15. Handling NULL values in MySQL

1. Using IS NULL and IS NOT NULL:

To check for NULL values in your queries, you can use the IS NULL and IS NOT NULL operators.

SELECT * FROM table_name WHERE column_name IS NULL;

2. Using COALESCE:

The COALESCE function returns the first non-NULL value from the list of arguments provided.

-- Replace NULL with 'Default Value'

SELECT COALESCE(column_name, 'Default Value') AS column_name FROM table_name;

3. Using IFNULL:

The IFNULL function returns a specified value if the expression is NULL, otherwise, it returns the expression.

-- Replace NULL with 0

```
SELECT IFNULL(column_name, 0) AS column_name FROM table_name;
```

17. Difference between IN and EXISTS?

- IN is faster for a small set.
- EXISTS is better for large data.
- the IN clause checks if a value matches any value in a list or subquery.
- The EXISTS clause checks if a subquery returns any records. It returns TRUE if the subquery returns one or more records, and FALSE if it returns no records.

```
SELECT * FROM employees WHERE dept_id IN (1,2,3);
```

```
SELECT * FROM employees WHERE EXISTS (SELECT 1 FROM departments WHERE id = employees.dept_id);
```

18. What is a subquery? Example of correlated subquery.

A subquery is a query nested inside another SQL query. It is used to retrieve data that will be used in the main query.

Types of Subqueries:

1. Single-row subquery – Returns a single value.
2. Multi-row subquery – Returns multiple values.
3. Correlated subquery – Depends on the outer query and executes once per row.

```
SELECT name FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
```

Correlated subqueries use values from outer queries.

19. What is a MySQL transaction?

A transaction in MySQL is a set of SQL statements that are executed as a single unit of work. Transactions ensure data consistency and integrity by following the ACID properties (Atomicity, Consistency, Isolation, Durability).

◆ ACID Properties of Transactions

1. Atomicity – Ensures that all operations in a transaction either succeed completely or fail completely.
2. Consistency – Guarantees that the database remains in a valid state before and after a transaction.

- 3.Isolation – Ensures that transactions do not interfere with each other.
- 4.Durability – Once a transaction is committed, the changes are permanently saved.

START TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE id=1;

UPDATE accounts SET balance = balance + 500 WHERE id=2;

COMMIT;

Rollback if needed:

ROLLBACK;

19. Creating and using stored procedures

DELIMITER //

CREATE PROCEDURE GetUsers()

BEGIN

 SELECT * FROM users;

END //

DELIMITER ;

CALL GetUsers();

20. Purpose of an INDEX?

An INDEX in MySQL is used to speed up query execution by allowing the database to locate data quickly instead of scanning the entire table.

♦ Why Use an Index?

- ✓ Faster Data Retrieval – Indexes reduce search time by creating an efficient lookup structure.
- ✓ Efficient Sorting & Filtering – Helps in ORDER BY, WHERE, and JOIN queries.
- ✓ Improves Performance – Reduces the number of records MySQL scans for a query.

♦ Types of Indexes in MySQL

- 1.Primary Index – Automatically created for the PRIMARY KEY column.
- 2.Unique Index – Ensures values in the column are unique.
- 3.Composite Index – Created on multiple columns for multi-condition searches.

4.Full-Text Index – Used for text searching in large text fields.

5.Spatial Index – Used for geographical data.

21. What are MySQL views?

A virtual table:

```
CREATE VIEW high_salary AS SELECT * FROM employees WHERE salary > 60000;
```

22. Implementing full-text search

sql

CopyEdit

```
ALTER TABLE articles ADD FULLTEXT(title, body);
```

```
SELECT * FROM articles WHERE MATCH(title, body) AGAINST ('MySQL');
```

23. MyISAM vs. InnoDB?

- **MyISAM:** Fast, no transactions.
- **InnoDB:** Slower, supports transactions.

24. Designing schemas for performance

Use normalization to avoid redundancy, but avoid over-normalization for performance.

25. ACID properties in MySQL

Ensured by InnoDB engine.

26. Query optimization techniques

Use EXPLAIN, indexes, and avoid SELECT *.

27. Materialized views simulation

Use indexed tables.

28. MySQL replication types

Master-Slave Replication:

- **Description:** In this configuration, one server (the master) writes data, and one or more servers (the slaves) read data. Changes made on the master are asynchronously copied to the slaves.
- **Master-Master Replication:**

Description: Also known as bidirectional replication, both servers can act as a master and a slave, allowing changes to be made on either server. The changes are then synchronized between the servers.

Multi-Master Replication:

- **Description:** Extends master-master replication to multiple servers, where each server acts as both a master and a slave. All changes are synchronized among all servers.

29. Managing concurrency

Use LOCK TABLES, row-level locking.

30. Sharding in MySQL

Distributes data across servers to scale performance.